Programación Avanzada

Tema 2. Relaciones entre clases

Índice

Esquema

Ideas clave

2.1. Introducción y objetivos

2.2. Relaciones entre clases

2.3 Cardinalidad/multiplicidad de una relación

2.4 Navegabilidad de una asociación

2.5. Tipos de relaciones entre clases

2.6 Cuaderno de ejercicios

A fondo

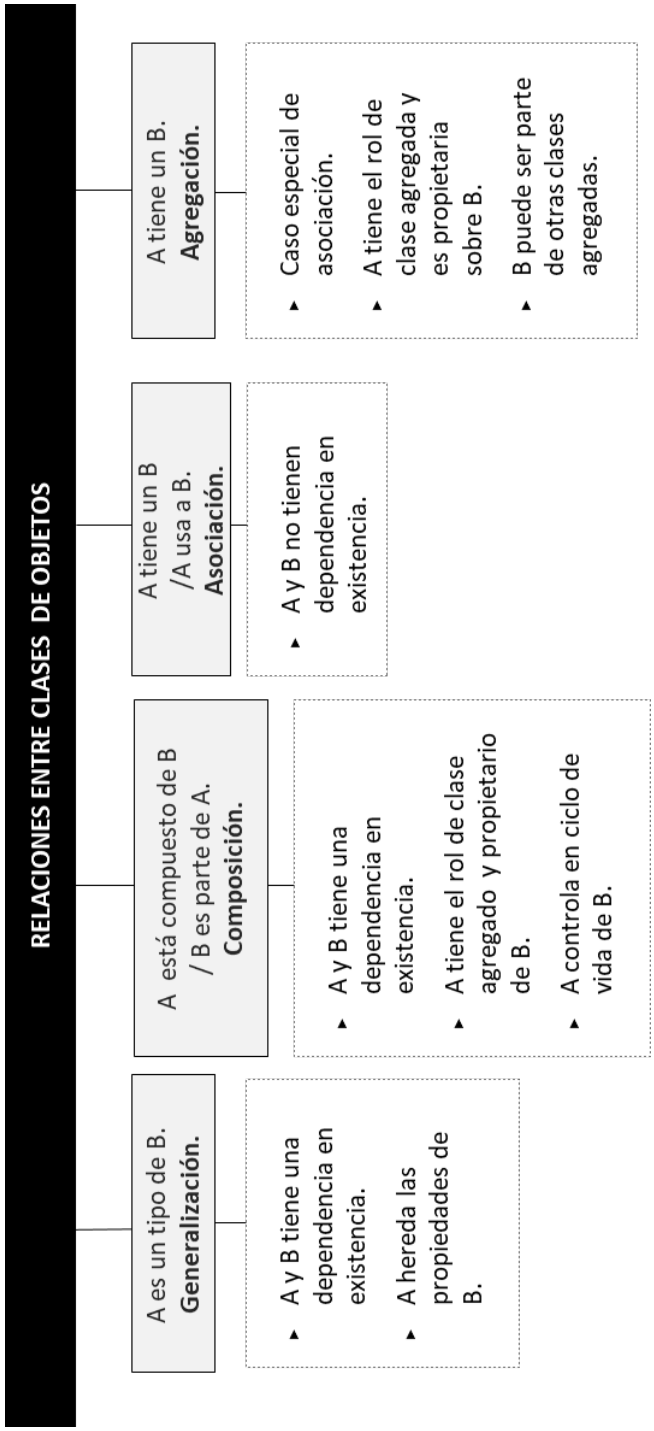
Catálogo de refactorizaciones

Measuring coupling and cohesion in object-oriented systems

Los principios SOLID

Programación orientada a objetos usando Java

Test



© Universidad Internacional de La Rioja (UNIR)

2.1. Introducción y objetivos

En un modelo orientado a objetos, será difícil modelar clases de objetos sin tener en cuenta algún tipo de asociación entre ellos. Surge, por tanto, la necesidad de ver y estudiar qué tipos de relaciones se pueden producir en el contexto del diseño y modelado de clases de objetos.

El objetivo fundamental de este tema es conocer los distintos tipos de relaciones entre las clases, como son: la relación de uso, relación de agregación (débil y fuerte) y relación de generalización/especialización. En este tema, particularmente, nos centraremos en las dos primeras.

Los **objetivos generales** de este tema son los siguientes:

- Estudiar y comprender los distintos tipos de relaciones que se pueden dar entre clases.
- Estudiar como implementar relaciones entre clases.
- Comprender como implementar restricciones de cardinalidad y navegabilidad sobre asociaciones.

© Universidad Internacional de La Rioja (UNIR)

2.2. Relaciones entre clases

Es difícil concebir un dominio de un problema de programación orientada a objetos donde las entidades conceptuales que se presentan para modelar no presenten algún tipo de relación entre ellas. **Las relaciones conceptuales surgirán de forma natural al estar todas esas clases de objetos dentro de un mismo dominio conceptual.** Así, por ejemplo, en un dominio de información académico, la entidad Estudiante tendrá algún tipo de relación con conceptos tales como Asignatura, Curso, o Grupo, etc. ¿Cómo se modelan estas relaciones?

El concepto de relación entre objetos tiene connotación colaborativa, desde el punto de vista funcional.

Si se considera un objeto como una entidad dinámica dotada de capacidad de proceso, los objetos se asocian y colaboran para alcanzar algún fin de procesamiento conjunto, y lo harán intercambiando mensajes entre ellos

En el siguiente ejemplo (Figura 1), se muestra una colaboración donde un objeto de t i p o **MotorTopología** se apoya en dos objetos: **ProcesadorLineas** y **ProcesadorPoligonos** para extraer y procesar elementos de capas vectoriales.

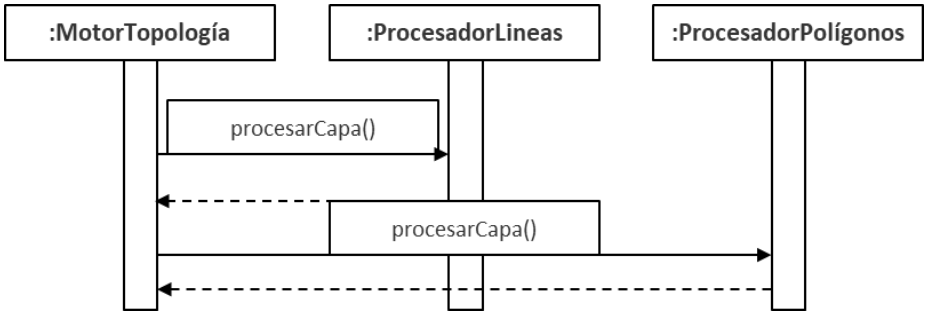
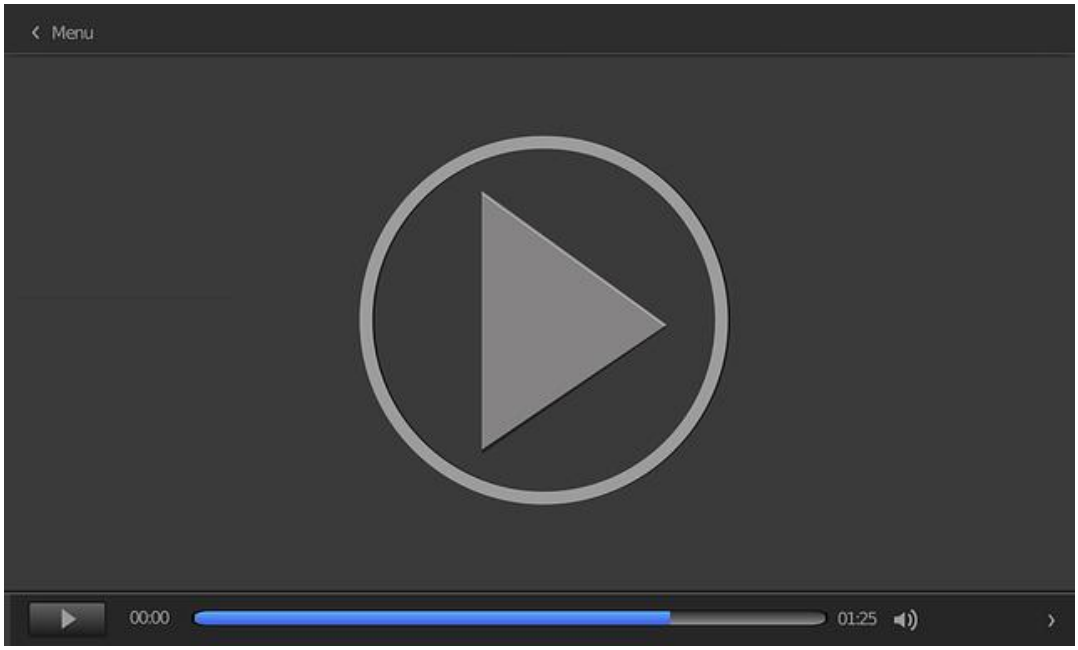


Figura 1. Diagrama de secuencia mostrando una colaboración entre objetos. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Los diagramas de secuencia son esenciales para especificar la secuencia de una interacción entre objetos y, por tanto, la perspectiva dinámica de una parte del sistema. En este tipo de diagrama UML, se pone el foco en cuestiones tales como la secuencia de intercambio de mensajes entre objetos, su orden cronológico y el momento en que cada objeto interviene. Esto lo puedes ver en el siguiente vídeo *Diagramas de secuencia UML*:



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=593c84ee-9710-4897-bd36-aff600d6cc5f

De forma general y sin entrar en los distintos tipos de relaciones que existen, podemos decir que dadas dos clases A y B, una asociación entre A y B representa algún tipo de relación semántica existente entre dichas clases en el dominio del problema. La asociación se representa gráficamente en UML con un segmento que une las dos clases implicadas en la relación.

© Universidad Internacional de La Rioja (UNIR)

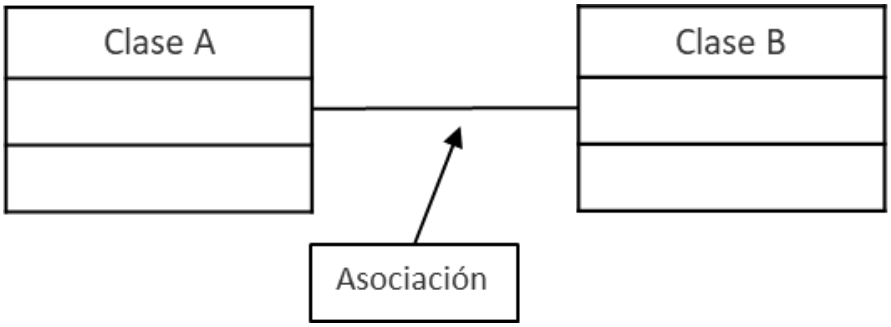


Figura 2. Representación en UML de una asociación entre clases. Fuente: elaboración propia.

Podemos enriquecer todavía más la semántica de la relación, etiquetándola con los roles que desempeñan cada una de las clases que participan en la relación y añadiendo las cardinalidades de la relación para cada extremo de la asociación.

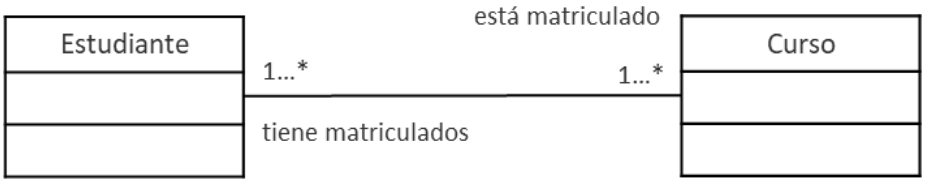


Figura 3. Asociación entre clases: Estudiante y Curso. Fuente: elaboración propia.

Una posible instancia de estos objetos y sus relaciones podría ser la siguiente:

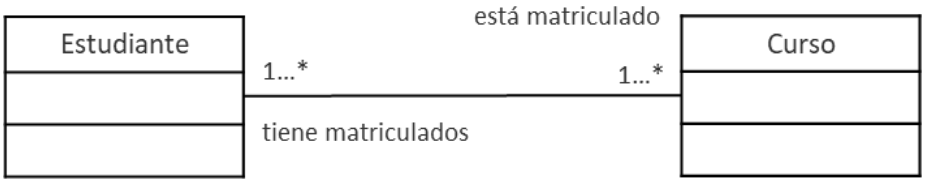


Figura 4. Representación de objetos relacionados. Fuente: elaboración propia.

Aquí vemos, por ejemplo, como el tipo de objeto Estudiante Pedro está relacionado con dos objetos de tipo de Curso: Seguridad e Ingeniería sw.

© Universidad Internacional de La Rioja (UNIR)

Otros ejemplos de asociaciones entre clases (ver siguiente Figura):

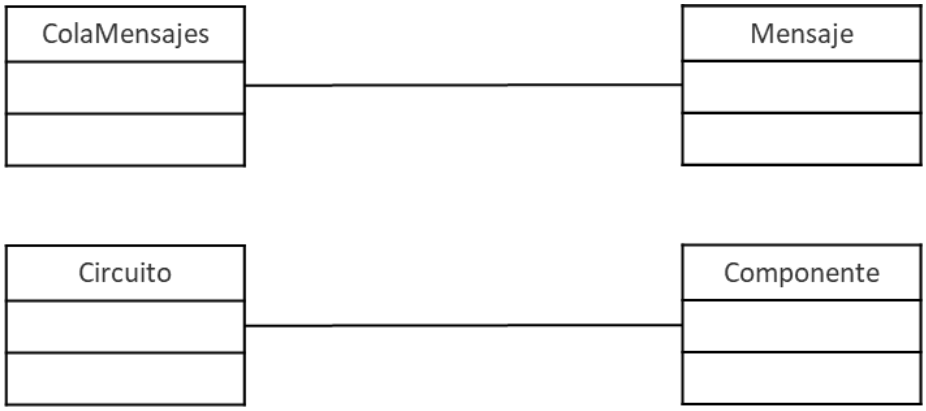


Figura 5. Ejemplos de Clases relacionadas. Fuente: elaboración propia.

Al modelar tipos de objetos (clases) y sus relaciones, también **podemos representar restricciones sobre el número de instancias que pueden participar en la asociación (multiplicidad de la asociación).** Por ejemplo, al observar los ejemplos anteriores, surgen las siguientes cuestiones: un objeto de tipo Mensaje ¿con cuántos objetos de tipo ColaMensajes está relacionado y viceversa? O, de la misma forma, ¿con cuántos objetos de tipo Circuito puede estar relacionado un objeto de tipo Componente?

Implementación de la asociación entre clases

Vamos a ver como implementar una asociación entre dos clases en el lenguaje de programación Java, para ello consideramos el siguiente modelo de clases (ver siguiente Figura).

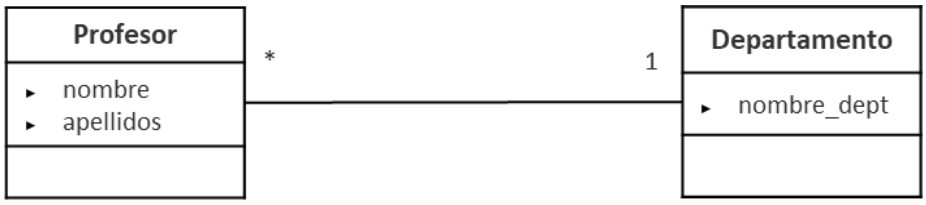


Figura 6. Modelo de clases representado una asociación. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

La semántica de esta relación es, según el modelo, la siguiente: un profesor estará relacionado con un objeto de tipo de Departamento y un Departamento puede estar relacionado con varios objetos de tipo Profesor.

El siguiente código muestra cómo, estructuralmente, **la clase Profesor tiene una referencia a un objeto de tipo Departamento.** Por restricción del modelo, está debe ser única y luego se modela con una única referencia a un objeto de tipo Departamento. En el sentido inverso, la clase Departamento está relacionada con N (indicado en UML como «*») Profesores, luego, **estructuralmente, la clase Departamento debe poder hacer referencia a N referencias de objetos tipo Profesor.** Esto lo hará mediante un tipo de datos-colección que, en este caso, en Java puede ser un objeto de tipo ArrayList:

```
public class Profesor {

//Atributos de clase

String nombre;

String apellidos;

//Métodos

Departamento dept;

Profesor (){…}

}



public class Departamento {

//Atributos de clase
```


© Universidad Internacional de La Rioja (UNIR)

```
String nombre_dept;

ArrayList < Profesor> empleados;

//Métodos

Departamento(){…}

public void setProfesor(Profesor p){

empleados.add(p);

}

}
```

Como hemos visto en el ejemplo anterior, una relación de una clase con otra se codifica en la declaración de la clase mediante referencias a un tipo de objeto de la otra clase.

© Universidad Internacional de La Rioja (UNIR)

2.3 Cardinalidad/multiplicidad de una relación

Una asociación entre clases representa una interconexión entre instancias de objetos de dos clases, pero —en numerosos casos— esa asociación estará restringida en el número de elementos que pueden participar de ella. Por ejemplo, un Profesor podría pertenecer a un solo Departamento, una Cuenta Bancaría podría estar restringida a tener como mínimo un Titular o como máximo dos ¿Cómo expresamos estás restricciones?

Para estos casos, es necesario definir la cardinalidad (o multiplicidad) de las instancias que participan en la relación como restricción.

La cardinalidad/multiplicidad es el número de objetos de un extremo de la asociación que están enlazados con un objeto del otro extremo.

La cardinalidad se representa como un rango o valor concreto en los extremos de la asociación.

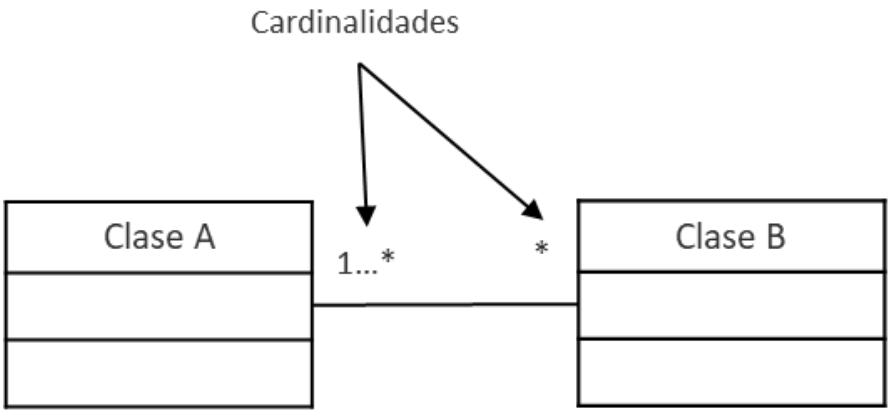


Figura 7. Representación de cardinalidades. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

La cardinalidad es una de restricción introducida en el modelo y sirve para representar más fielmente la realidad del problema (por ejemplo, un estudiante solo puede estar matriculado en un único curso).

La cardinalidad en UML puede ser expresada de la siguiente forma:

- **Número exacto (1, 3, etc.):** expresa que un objeto de la clase Clase A tiene que estar relacionado obligatoriamente con el número de objetos que indica la cardinalidad de la clase Clase B.

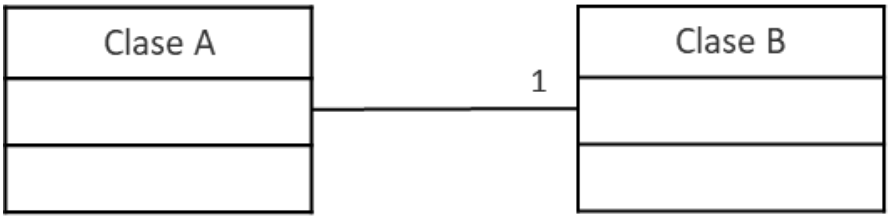


Figura 8. Número exacto. Fuente: elaboración propia.

- **Rango de valores (0, 1, 3, … 5, etc.):** con esta notación indicamos el número mínimo y máximo de objetos con los que puede estar relacionado un objeto en el otro extremo de la relación. En el ejemplo, un objeto instancia de la Clase A podrá estar relacionado como mínimo con una instancia de la Clase B y como mucho con tres instancias.

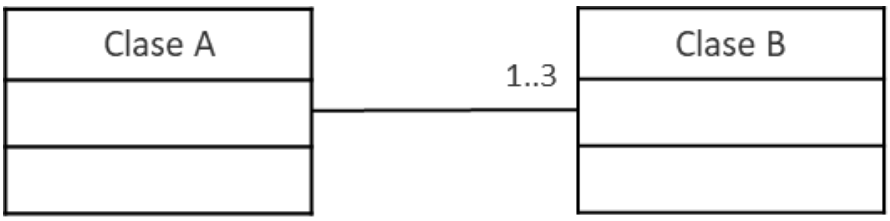


Figura 9. Rango de valores. Fuente: elaboración propia.

- **Múltiples (*):** indica que la instancia de la clase Clase A puede estar relacionada con cualquier número de instancias de la clase Clase B (o incluso con ninguna). Equivale a escribir **0..*.**

© Universidad Internacional de La Rioja (UNIR)

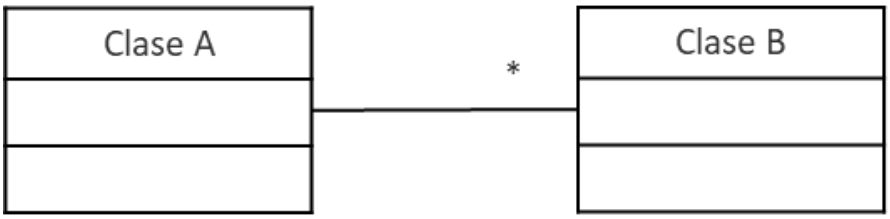


Figura 10. Múltiples. Fuente: elaboración proia.

- **Combinaciones de las anteriores.** A continuación se muestran algunos ejemplos de cardinalidades.

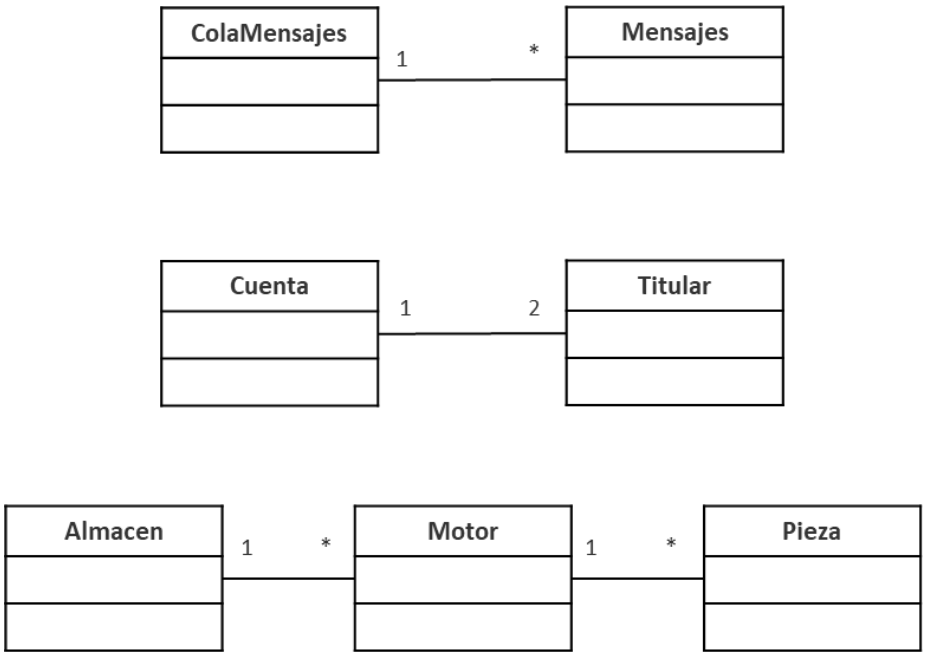


Figura 11. Ejemplo de multiplicidades/cardinalidades. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

2.4 Navegabilidad de una asociación

Otra característica (o restricción) que podemos expresar en una relación entre clases **es la navegabilidad de la relación**. La navegabilidad de la relación **indica el sentido en el cuál es posible recorrer la asociación.**

Las dos opciones o tipos de navegabilidad posibles son las siguientes:

- **Asociación unidireccional:** solo una de las dos clases tiene constancia de la existencia de la otra. Y, consecuentemente, solo un objeto de la relación representará la relación con el otro tipo de objeto.

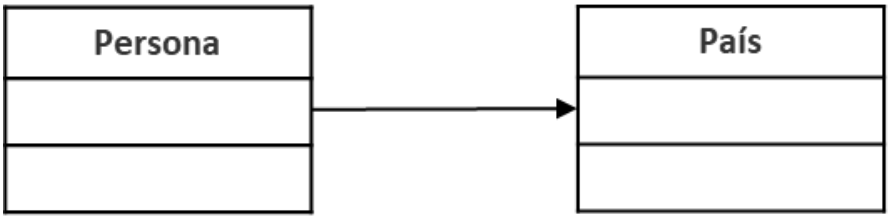


Figura 12. Asociación unidireccional. Fuente: elaboración propia.

La interpretación semántica de esta restricción es la de que solo un tipo de objeto va a interpretar la relación, por lo que, estructuralmente, aparece la referencia de la relación en el objeto que interpreta la relación.

- **Asociación bidireccional:** los dos tipos de objetos pueden interpretar la relación indistintamente.

© Universidad Internacional de La Rioja (UNIR)

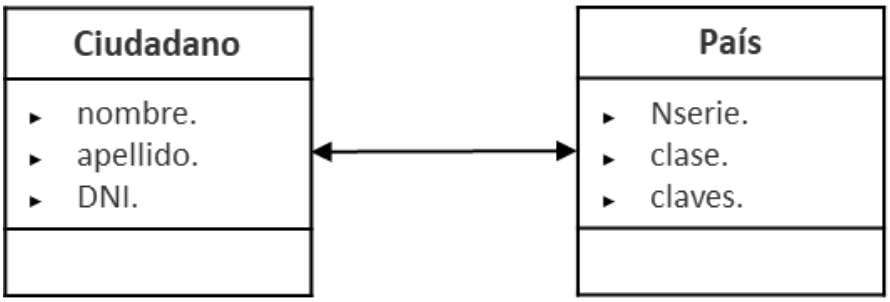


Figura 13. Asociación bidireccional. Fuente: elaboración propia.

Vamos a ver ahora como podríamos implementar la navegabilidad en un lenguaje de programación.

Implementación de la relación unidireccional en Java

Dada la siguiente clase en Java:

```
public class Persona {

//Atributos de clase

private String nombre;

private String apellidos;

private Departamento dept;//Relación con objeto Departamento

//Métodos



public Persona (Departammento d){

dept=d;

};

}
```


© Universidad Internacional de La Rioja (UNIR)

```
public class Departamento {

private String nombre;

}
```

Como se puede observar, en la implementación anterior, desde un objeto de tipo Persona se tendrá acceso a uno de tipo Departamento, pero no al revés. La asociación, por tanto, es en una única dirección desde Persona a Departamento.

Implementación de la relación bidireccional en Java

```
public class Ciudadano {

//Atributos de clase

private String nombre;

private String apellidos;

private String dni;//Relación con objeto Departamento

private CertificadoDigital cert;

}

public class CertificadoDigital {
```

- private String nserie;
- private int clase_cert;

© Universidad Internacional de La Rioja (UNIR)

- private String claves;
- **private Ciudadano ciudadano;**
- }

En este caso, sin embargo, ambos tipos de objetos tienen acceso a la asociación.

© Universidad Internacional de La Rioja (UNIR)

2.5. Tipos de relaciones entre clases

Como hemos visto, una relación representa cualquier tipo de conexión semántica entre dos clases. Los tipos de relaciones pueden ser clasificadas de forma informal de la siguiente manera:

- **Relación de asociación:** utilidad o de servicio.
- **Relación de pertenencia o jerarquía:**
- **Relación «es parte-de»:** agregación débil o fuerte y de generalización.
- **Relación «es un tipo de»:** generalización /especialización.

Si dos clases necesitan comunicarse entre sí, debe haber un vínculo entre ellas, y eso puede representarse mediante una asociación.

A continuación, se muestra las convenciones UML empleadas para cada tipo de relación anteriormente expuesta:

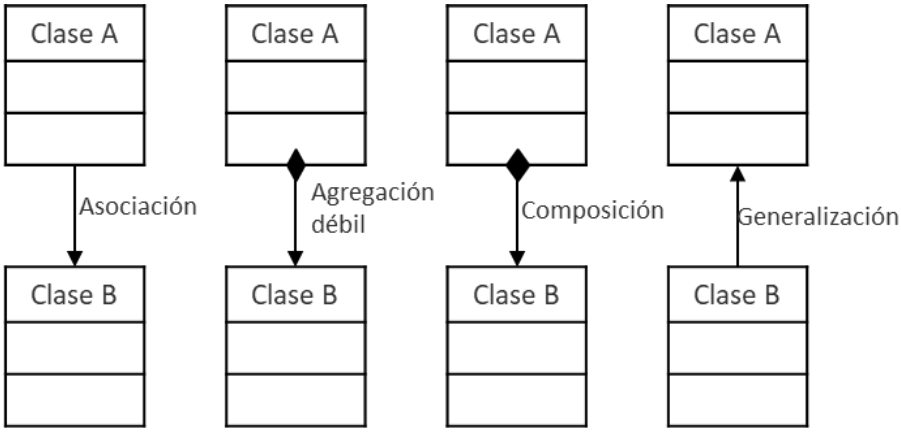


Figura 14. Tipos de relaciones entre clases. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Al modelar este **tipo de relaciones,** surge la cuestión de **cómo diferenciarlas.** Un **criterio es el de la dependencia en existencia.** Es decir, si un tipo de objeto depende de la existencia de otro tipo. Si esta dependencia es clara, estamos ante una relación de pertenencia y deberemos plantearnos si modelarla como una agregación débil o fuerte (composición). Veremos estos importantes conceptos a continuación.

Relación de asociación

En un domino modelado con clases de objetos, existen muchos escenarios donde dos clases de objetos se pueden asociar para llevar a cabo algún tipo de proceso coordinado.

Un escenario común es aquel donde una clase necesita el servicio que otra clase proporciona. Desde este punto de vista de **un objeto,** se nos presenta como un **proveedor de algún tipo de servicio que otro objeto utiliza.** Por ejemplo, un Controlador HTTP (ServletController) en el contexto de un servidor Web lleva a cabo su función apoyándose en otras clases que le aportan algún servicio, como la clase JavaServerPage o ServletContext. La clase ServletController no tiene una relación de jerarquía y pertenencia con estas dos clases, sino una relación de uso de los servicios que le proporcionan las otras dos clases. El siguiente modelo de clases muestra esta relación.

© Universidad Internacional de La Rioja (UNIR)

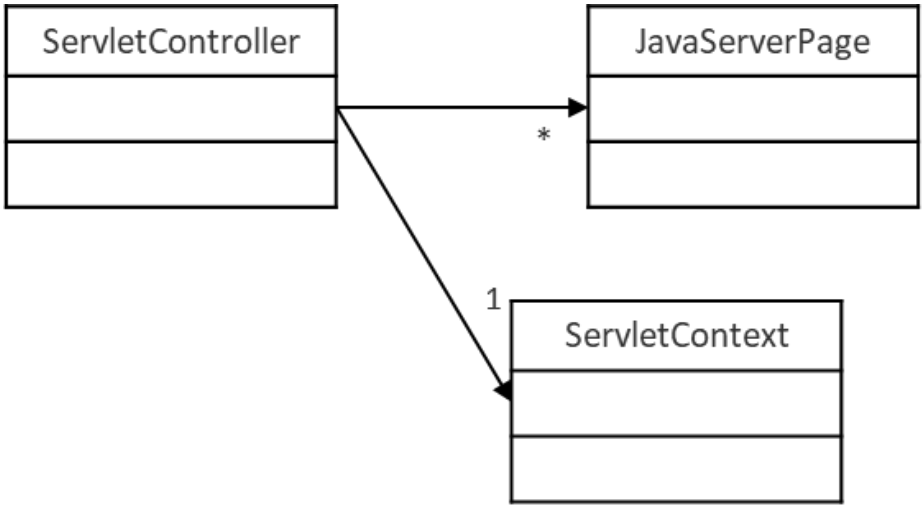


Figura 15. Relación de uso desde la clase ServletController hacia la clase JavaServerPage y ServletContext. Fuente: elaboración propia.

La asociación de uso se suele presentar de la siguiente forma:

- Una clase recibe como parámetro de uno de sus métodos a otra clase (clase que proporciona el servicio).
- Una clase crea una instancia de otro tipo de objeto en uno de métodos (el alcance de esa instancia y su unión es el método).

© Universidad Internacional de La Rioja (UNIR)

Gráficamente, la relación de uso se representa con un segmento (ver Figura 16).

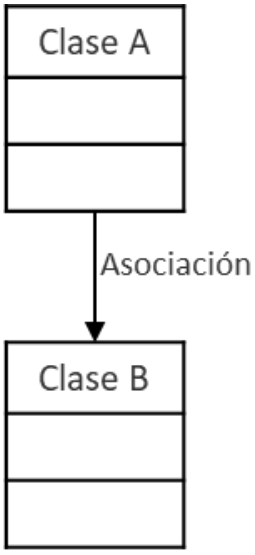
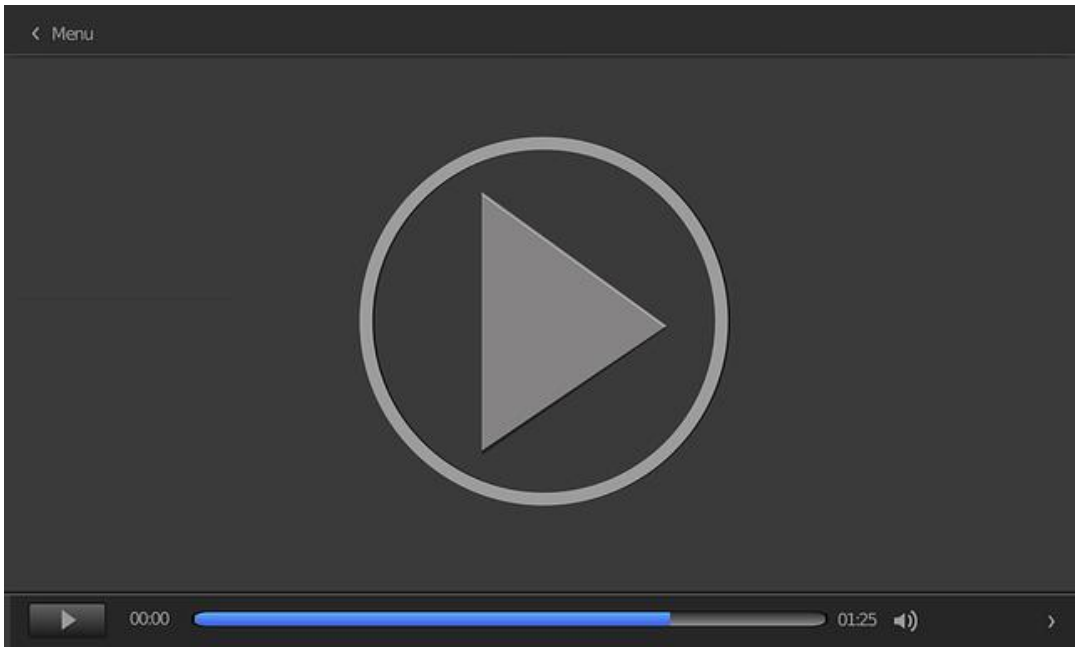


Figura 16. Representación de una relación de uso entre clases. Fuente: elaboración propia.

Esta relación es muy común. La relación es puramente de utilidad y se caracteriza por que la entidad conceptual representada por la Clase A necesita algo de la Clase B, pero solo a nivel de comportamiento.

© Universidad Internacional de La Rioja (UNIR)

A continuación, puedes ver el vídeo *Asociaciones bidireccionales.*



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=2c4d2491-0802-41a5-bb1a-aff600df5a8a

Asociación reflexiva

La asociación reflexiva es un tipo de relación de asociación en la que las instancias de la misma clase se pueden asociar entre sí. Por ejemplo, al representar una estructura de directorios en un sistema de archivos.

© Universidad Internacional de La Rioja (UNIR)

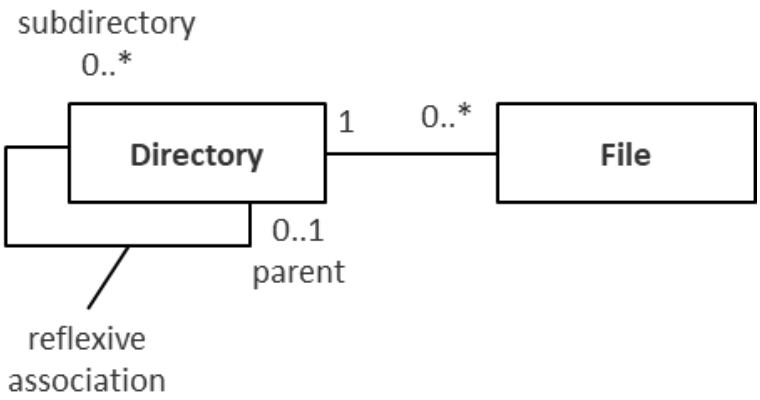


Figura 17. Asociación reflexiva. Fuente: elaboración propia.

La asociación reflexiva indica que el vínculo o la conexión denota al mismo tipo de clase.

Relación de agregación

La agregación es una forma especial de asociación. La relación de agregación surge cuando un objeto queda caracterizado a partir de la agrupación de otros objetos en su estructura. Corresponde a la concepción de una entidad caracterizada por partes y responde a la semántica: **un objeto B es parte de A** o, de manera inversa, **A tiene un B.** Donde A representa el objeto agregación y B sería una parte de esa agregación.

En función del grado de dependencia del objeto-parte respecto al todo, podemos diferenciar dos tipos de agregación: **agregación débil y agregación fuerte.**

Agregación débil o agregación compartida

La asociación de agregación débil modela la relación semántica todo/parte, o bien A tiene un B o B es parte de A. La cualidad de débil se da cuando la parte puede formar parte de otras relaciones o B no depende en existencia de A.

© Universidad Internacional de La Rioja (UNIR)

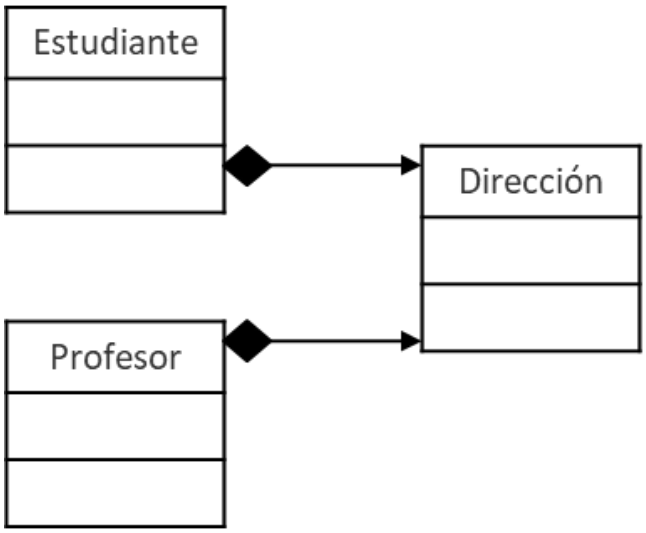


Figura 18. Agregación débil. Fuente: elaboración propia.

La representación de esta relación se realiza en UML mediante un rombo en el extremo de la clase que representa el **objeto agregado.**

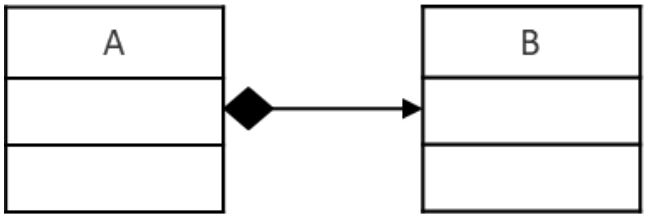


Figura 19 Representación en UML de una agregación débil. Fuente: elaboración propia.

El rol **parte** puede pertenecer a más de un agregado (de ahí el nombre de agregación compartida o débil). Además, la existencia de los objetos con el rol de parte no está supeditada a la existencia del objeto **todo.**

Agregación de composición

Una agregación de composición es aquella relación entre clases en la que se concibe el **todo como un objeto compuesto de objeto-parte** y los objetos-parte tienen una dependencia en existencia del agregado.

© Universidad Internacional de La Rioja (UNIR)

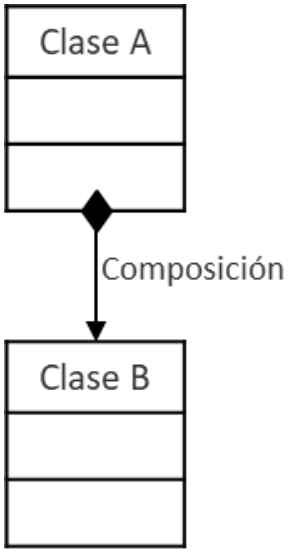


Figura 20. Representación UML de una composición. Fuente: elaboración propia.

La agregación fuerte confiere a la asociación la característica de que el **todo** no puede existir sin las partes ni viceversa (no tiene sentido que los **objeto-parte** existan sin el **todo**). El ciclo de vida de las **partes** está estrechamente ligado al del **todo,** de forma que si este se destruye, se destruyen también las **partes.**

Tómese el siguiente ejemplo (ver Figura 21) que representa el esquema de definición del concepto de **patrón software** como ejemplo de modelado de una composición.

© Universidad Internacional de La Rioja (UNIR)

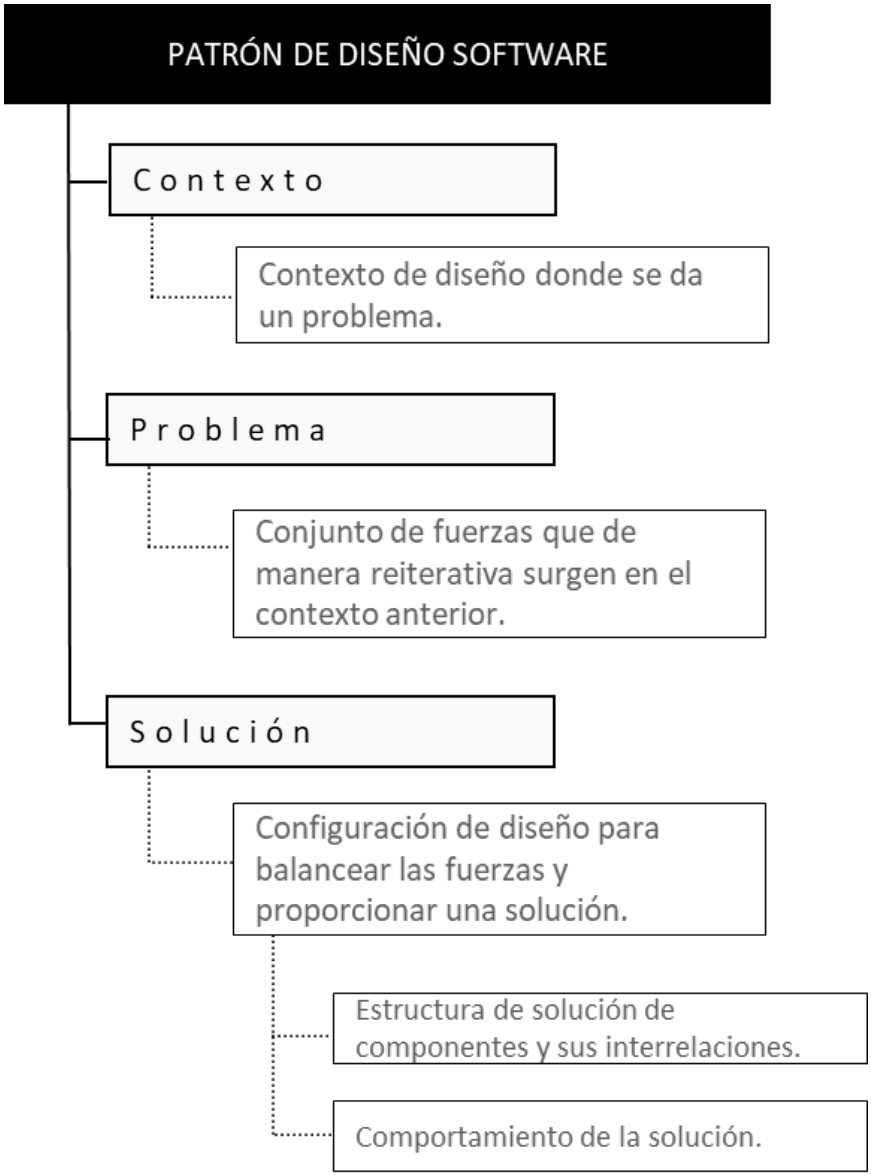


Figura 21. Ejemplo de esquema candidato a modelarse como una relación de composición. Fuente: elaboración propia.

En el ejemplo anterior, se muestra la definición del concepto de patrón de diseño software como un esquema de tres partes**: el contexto, el problema y la solución.** Si quisiéramos modelar un sistema de información que procese el concepto de patrón de diseño, podríamos hacerlo mediante la siguiente relación de composición de clases.

© Universidad Internacional de La Rioja (UNIR)

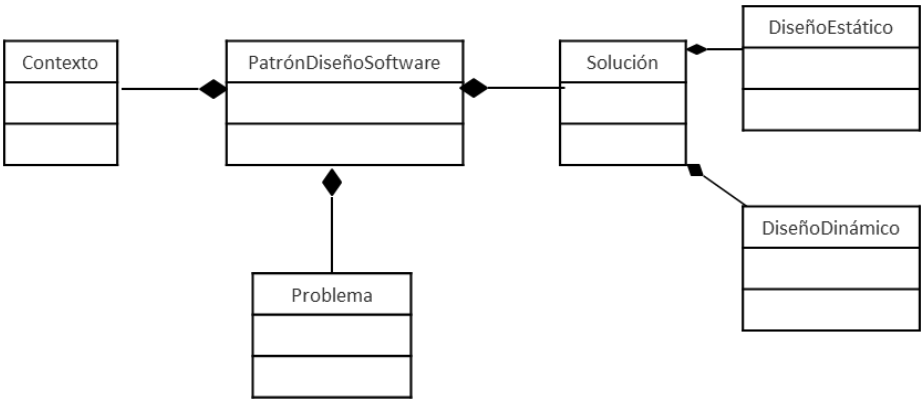


Figura 22. Ejemplo del concepto de patrón de diseño software modelado como composición de clases. Fuente: elaboración propia.

En el ejemplo anterior, la clase PatrónDiseñoSoftware representa el todo y las clases Contexto, Problema y Solución representan las partes de la composición. Existe, además, una relación de dependencia en existencia, ya que si el patrón desaparece, el Problema, el Contexto y la Solución no tienen sentido fuera de él. Esta relación responde a las restricciones impuestas por una composición.

A diferencia de la agregación simple, la agregación de composición requiere que las partes no puedan tener una relación de agregación con a otra clase (no pueden estar compartidas).

© Universidad Internacional de La Rioja (UNIR)

2.6 Cuaderno de ejercicios

1. Dadas las siguientes relaciones en el contexto de una misión lunar, indique cuáles deberían modelarse como composición:

- Relación
- Módulo lunar – Motor.
- Astronauta – Traje espacial.
- Base lunar – Sistema de soporte vital.
- Rover lunar – Cámara científica.
- Misión lunar – Experimentos.

Solución

Se modelan como composición aquellas relaciones donde el objeto contenido no puede existir sin el contenedor:

✔ Base lunar – Sistema de soporte vital

✔ Módulo lunar – Motor

No son composición:

✘ Astronauta – Traje espacial (puede reutilizarse)

✘ Rover lunar – Cámara científica (puede desmontarse)

✘ Misión lunar – Experimentos (relación más débil, agregación)

© Universidad Internacional de La Rioja (UNIR)

2. Indique el tipo de relación UML más adecuada en cada caso:

- Un objetivo científico lunar puede resolverse mediante uno o varios experimentos.
- Un informe de misión lunar es redactado por uno o varios científicos.
- Un rover lunar es asignado a una misión lunar concreta.

Solución

- Objetivo – Experimento → Asociación.
- Informe – Científico → Asociación muchos a muchos.
- Rover – Misión lunar → Agregación (el rover puede usarse en otras misiones).

3. Dado el siguiente fragmento de código:

```
public class ControladorMisionLunar {

    public void calcularConsumo() {

        double combustible = Double.parseDouble(textCombustible.getText());

        ConversorUnidades conv = new ConversorUnidades();

        double consumoKg = conv.convertirALibras(fuel);

    }

}
```

- ¿Qué tipo de relación existe entre ControladorMisionLunar y ConversorUnidades?
- Razone la respuesta.

© Universidad Internacional de La Rioja (UNIR)

Solución

La relación es de **uso (dependencia)**.

El objeto ConversorUnidades:

- Se crea dentro del método.
- Solo se utiliza durante la ejecución del mismo.
- No forma parte del estado del controlador.

4. Una nave lunar está formada por:

- Unidad de control
- Sistema de propulsión
- Sistema de navegación
- Módulo de comunicación
- Sistema de energía

Modele esta arquitectura mediante composición.

Solución (descripción UML)

NaveLunar

- UnidadControl
- SistemaPropulsion
- SistemaNavegacion

© Universidad Internacional de La Rioja (UNIR)

- SistemaComunicacion
- SistemaEnergia

✔ Todos los componentes dependen completamente de la nave.

5. Una trayectoria lunar está definida por dos puntos en el espacio.

Dada la clase PuntoEspacial, complete la clase Trayectoria.

```
public class PuntoEspacial {

        private double x;

        private double y;

        private double z;



        public PuntoEspacial(double x, double y, double z) { ... }

        public double getX() { ... }

        public double getY() { ... }

        public double getZ() { ... }

}
```

Solución

```
public class Trayectoria {



    private PuntoEspacial p1;

    private PuntoEspacial p2;
```


© Universidad Internacional de La Rioja (UNIR)

```
    public Trayectoria(PuntoEspacial p1, PuntoEspacial p2) {
        this.p1 = p1;
        this.p2 = p2;
    }

  public Trayectoria(double x1, double y1, double z1,
                     double x2, double y2, double z2) {
    p1 = new PuntoEspacial(x1, y1, z1);
    p2 = new PuntoEspacial(x2, y2, z2);
  }
}
```

6. ¿Cómo puede modelarse la relación del punto 5?

Solución

La relación puede modelarse como:

- **Composición** → si los puntos solo existen dentro de la trayectoria.
- **Agregación** → si los puntos pueden compartirse entre trayectorias.

✔ Ambas opciones son válidas según el dominio.

© Universidad Internacional de La Rioja (UNIR)

7. Analice el siguiente constructor:

```
public class RoverLunar {



        Motor motor;

        SistemaEnergia energia;

        Camara camara;



        public RoverLunar() {

                motor = new Motor("Eléctrico");

                energia = new SistemaEnergia("Solar");

                camara = new Camara(4K);

        }

}
```

Sabiendo que estos objetos no pueden modificarse desde fuera de la clase, indique el tipo de relación.

Solución

✔ Composición

© Universidad Internacional de La Rioja (UNIR)

El rover:

- Crea los componentes.
- Controla su ciclo de vida.
- No permite reutilización externa.

8. Una red topográfica lunar se forma mediante:

- Puntos lunares.
- Triángulos que conectan esos puntos.
- Modele este sistema mediante agregación.

Solución (descripción UML)

RedLunar

- PuntoLunar
- TrianguloLunar

✔ Los puntos pueden existir independientemente de la red.

9. Represente las siguientes relaciones:

A. Un módulo lunar puede tener como máximo dos brazos robóticos.

B. Una base lunar puede contener múltiples módulos y múltiples rovers.

C. Un objetivo científico se desarrolla mediante líneas de investigación, que a su vez se concretan en experimentos.

© Universidad Internacional de La Rioja (UNIR)

Solución

MóduloLunar (1) —— (0..2) BrazoRobotico

BaseLunar (1) —— (*) Modulo

BaseLunar (1) —— (*) Rover

Objetivo —— LíneaInvestigacion —— Experimento

10. Un Instituto de Investigación Lunar organiza su trabajo en equipos científicos.

Cada equipo:

- Está formado por mínimo 2 y máximo 5 investigadores.
- Pertenece a un único instituto.

Modele el sistema usando agregación y composición.

Solución

InstitutoLunar

- EquipoInvestigacion (composición).

EquipoInvestigacion

- Investigador (2..5) (agregación).

Los equipos no existen sin el instituto, pero los investigadores pueden cambiar de equipo.

© Universidad Internacional de La Rioja (UNIR)

Una posible solución sería

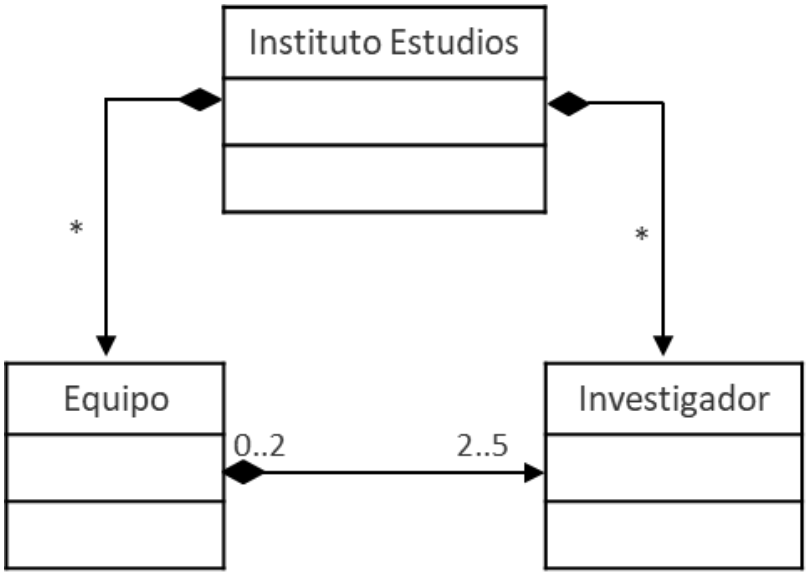


Figura 31. Solución ejercicio 10. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Catálogo de refactorizaciones

Refactoring. (s. f.). *Catalog.* https://refactoring.com/catalog/

La refactorización *(refactoring)* constituye una técnica, dentro de la ingeniería de software, para reestructurar un código fuente alterando su estructura interna sin cambiar su comportamiento externo. Algo importante es que la refactorización no corrige errores. El objetivo es mejorar la facilidad de comprensión del código o cambiar su estructura y diseño y eliminar código muerto para facilitar el mantenimiento en el futuro.

En esta web, del reconocido autor e ingeniero de software Martin Fowler, se publica un catálogo de casos de refactorización

© Universidad Internacional de La Rioja (UNIR)

Measuring coupling and cohesion in object-oriented systems

Hitz, M. y Montazeri, B. (1995). *Measuring Coupling and CohesionIn Object-Oriented Systems.* Institut für Angewandte Informatik und Systemanalyse, University of Vienna https://www.researchgate.net/publication/238729882_Measuring_coupling_and_cohesion_in_object-oriented_systems

El papel que juegan las métricas de software —en general— y el acoplamiento —en particular—, con respecto a la mantenibilidad de los productos de software, está ampliamente aceptado. En esta publicación se evalúan enfoques para manejar el acoplamiento o la cohesión en sistemas orientados a objetos.

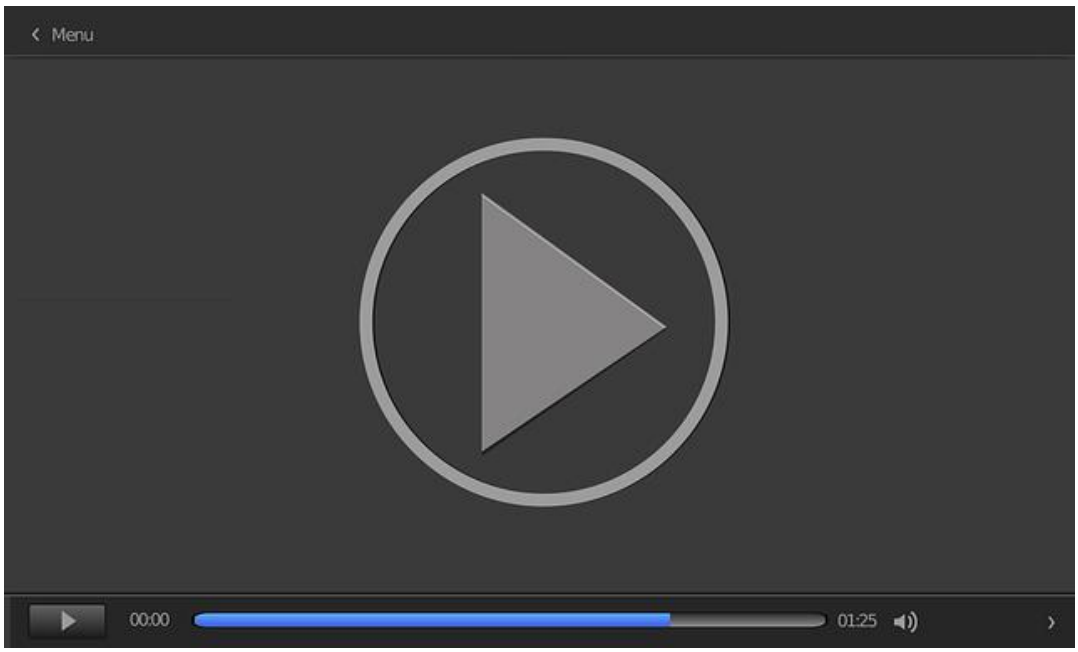
© Universidad Internacional de La Rioja (UNIR)

Los principios SOLID

BettaTech. (2020, febrero 10). *Los principios SOLID, ¡explicados!.* [Video]. YouTube. https://www.youtube.com/watch?v=2X50sKeBAcQ

En ingeniería de software, SOLID *(Single responsibility, Open-closed, Liskov substitution, Interface segregation and Dependency inversion)* es un acrónimo mnemónico introducido por Robert C. Martin que representa cinco principios básicos de la programación orientada a objetos y el diseño.

En este vídeo se explican los principios SOLID como guías que pueden ser aplicadas en el desarrollo de software. Los principios pretender guiar una estructura de diseño entre clases más fácil de mantener y más flexible a cambios



Accede al vídeo:

https://www.youtube.com/embed/2X50sKeBAcQ

© Universidad Internacional de La Rioja (UNIR)

Programación orientada a objetos usando Java

Flórez, H. A. (2012). Programación orientada a objetos usando Java. Ecoe Ediciones. https://www.academia.edu/50465126/Programaci%C3%B3n_orientada_a_objetos_usando_java_H%C3%A9ctor_A_Fl%C3%B3rez

Manual en el que se hace una introducción a la programación orientada objetos con Java.

© Universidad Internacional de La Rioja (UNIR)

1. Dos clases A y B están asociadas si:

   A. La clase A envía un mensaje a la clase B.

   B. La clase A crea una instancia de la clase B.

   C. La clase A recibe como parámetro de uno de sus métodos a la clase B.

   D. Todas las anteriores son correctas.

2. En UML la composición (o agregación fuerte) se representa:

   A. Con un triángulo apuntando a la clase hija.

   B. Con un triángulo apuntando a la clase padre.

   C. No existe este tipo relación.

   D. Con un rombo negro apuntando a la clase que representa la clase composición.

3. Una agregación es:

   A. La propiedad por la que una clase hereda comportamientos de otra clase llamada clase padre.

   B. Es la relación que tienen dos clases que se crean de manera consecutiva.

   C. Es la relación que tienen dos clases donde una clase forma parte de la otra.

   D. Es la relación que tienen dos clases que se intercambian mensajes.

4. En UML la agregación débil se representa:

   A. Con un triángulo apuntando a la clase hija.

   B. Con un triángulo apuntando a la clase padre.

   C. Con un rombo blanco en la clase que representa la agregación.

   D. Con un rombo negro en la clase hija.

© Universidad Internacional de La Rioja (UNIR)

5. Si un objeto depende de la existencia de otro objeto, estamos hablando de una relación de:

A. Composición.

B. Agregación débil.

C. Relación de uso.

D. Herencia.

6. La cardinalidad N en una relación entre clases se implementa mediante:

A. Una referencia.

B. Un tipo colección.

C. Un vector de tamaño fijo.

D. Ninguna de las anteriores.

7. Señale la respuesta correcta con respecto a la navegabilidad de una relación entre clases.

A. Siempre debe ser bidireccional.

B. Siempre debe ser unidireccional.

C. Puede ser bidireccional o unidireccional.

D. Ninguna de las anteriores.

8. Indique la respuesta correcta:

A. La cardinalidad y navegabilidad son restricciones de una relación.

B. La cardinalidad indica el número de instancias que pueden participar en una relación.

C. La navegabilidad implica restricciones en la interpretación de la relación.

D. Todas las anteriores son ciertas.

© Universidad Internacional de La Rioja (UNIR)

9. Indique la respuesta correcta sobre relaciones entre clases:

A. La relación de uso es la relación que produce menos acoplamiento entre clases.

B. La relación de composición es la relación que produce menos acoplamiento entre clases.

C. La relación de composición es menos restrictiva que la relación de agregación débil.

D. Todas las anteriores son falsas.

10. ¿Cómo se interpreta una cardinalidad 0..N en la relación entre la clase A respecto a la clase B?

A. Una instancia de la clase A puede estar asociada como mínimo con N instancias de la clase B.

B. Una instancia de la clase A puede estar asociada obligatoriamente con N instancias de la clase B.

C. Una instancia de la clase A debe estar asociada con N instancias de la clase B.

D. Una instancia de la clase A puede no estar asociada con ninguna de instancia de la clase B y como máximo puede estar asociada con N instancias.

© Universidad Internacional de La Rioja (UNIR)